Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Kapitel 03 - VHDL - Hardware-Beschreibungssprache

Prof. Dr.-Ing. Guillermo Payá Vayá, 12. November 2024

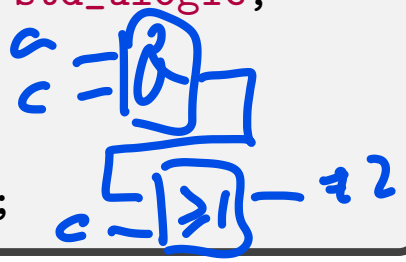
Nebenläufigkeit und Prozesse in VHDL

Nebenläufigkeit: Einleitung

THINK IN HARDWARE = THINK IN PARALLEL

- Hardware-Systeme arbeiten nicht sequentiell, sondern bestehen aus parallel arbeitenden, d.h. nebenläufigen, Teilsystemen
- Diese nebenläufigen Teilsysteme können entweder
  - eigenständige Einheiten sein (*entities*), die instanziiert werden können, um eine hierarchische Beschreibung des Systems zu erhalten,
  - sequentiell ausgeführte Prozesse darstellen oder
  - Anweisungen, die sich direkt im Körper der Architektur befinden
- Die Reihenfolge einzelner Anweisungen, Instanziierungen oder Prozesse hat daher keinen Einfluss auf das Verhalten

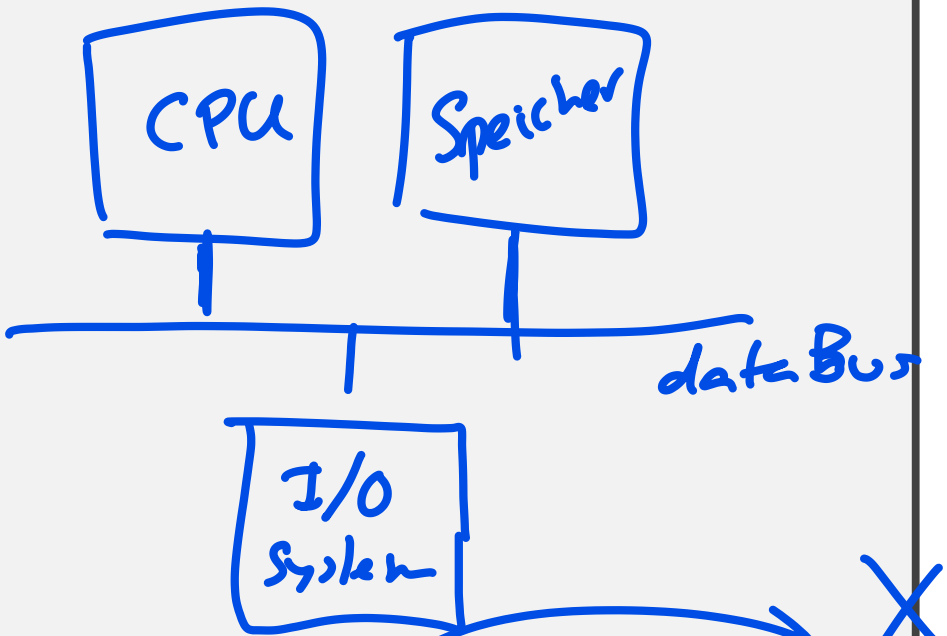
```
architecture behave of simpel is
  signal a, b, c : std_ulogic;
  signal z1, z2  : std_ulogic;
begin
  z1 <= a and c;
  z2 <= c or z1;
end process behave;
```




```
architecture behave of simpel is
  signal a, b, c : std_ulogic;
  signal z1, z2  : std_ulogic;
begin
  z2 <= c or z1;
  z1 <= a and c;
end process behave;
```

Nebenläufigkeit: Beispiel

```
architecture mixed of Mikrocomputer is
  signal dataBus : std_ulogic_vector(13 downto 0);
begin

  CPU_inst : CPU
    port map(
      data => dataBus,
      ...);

  Speicher: process (sList)
  begin
    ...
  end process Speicher;

  -- Zuweisungen fuer I/O
  dataIO <= dataBus when data_en = '1' else
            (others => 'Z');
  dataBuzs_r <= dataIO;
end architecture mixed;
```



z1 <= a;
z1 <= b;

multiple drive

Prozess: Einleitung

- Prozesse dienen der Modellierung des Verhaltens eines Systems
- Sie sind eine Liste sequentieller Anweisungen (vgl. "normale" Programmiersprachen)
- Syntax:

```
name: process (sensitivity_list)
  declarations
begin
  sequential_statements
end process name;
```

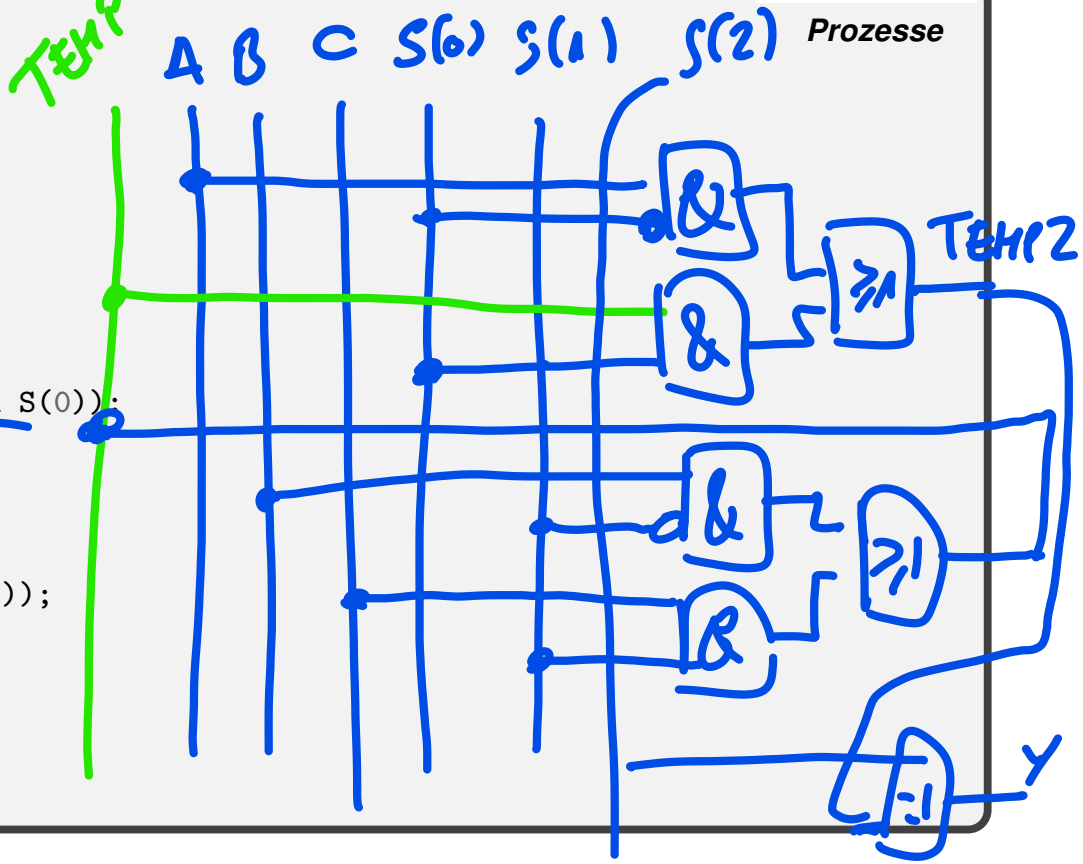
z1 ← c;
z1 ← b;

- `name`: Prozessname (optional, aber empfohlen)
- `sensitivity_list`: hier werden **alle** Signale eingetragen, deren Wertänderung die Ausführung des Prozesses bewirkt
- `declarations`: Hier werden Variablen und Konstanten deklariert
- `sequential_statements`: Alle sequentiell auszuführenden Prozesss-Anweisungen stehen zwischen `begin` und `end process`

Prozess: Beispiel

```
-- Einfache Schaltung mit 2 Prozessen und nebenl. Anweisung
-----------------------------------------------------------
entity BEISPIEL is
  port (A, B, C : in  std_ulogic;                    -- Dateneingang
        S       : in  std_ulogic_vector(2 downto 0); -- Selektionseingang
        Y       : out std_ulogic);                   -- Ausgang
end BEISPIEL;

architecture RTL of BEISPIEL is

  signal TEMP1, TEMP2 : std_ulogic;

begin

  MUX1: process (A, TEMP1, S(0))
  begin
    TEMP2 <= (A and not S(0)) or (TEMP1 and S(0));
  end process MUX1;

  MUX2: process (B, C, S(1))
  begin
    TEMP1 <= (B and not S(1)) or (C and S(1));
  end process MUX2;

  Y <= TEMP2 xor S(2);

end architecture RTL;
```

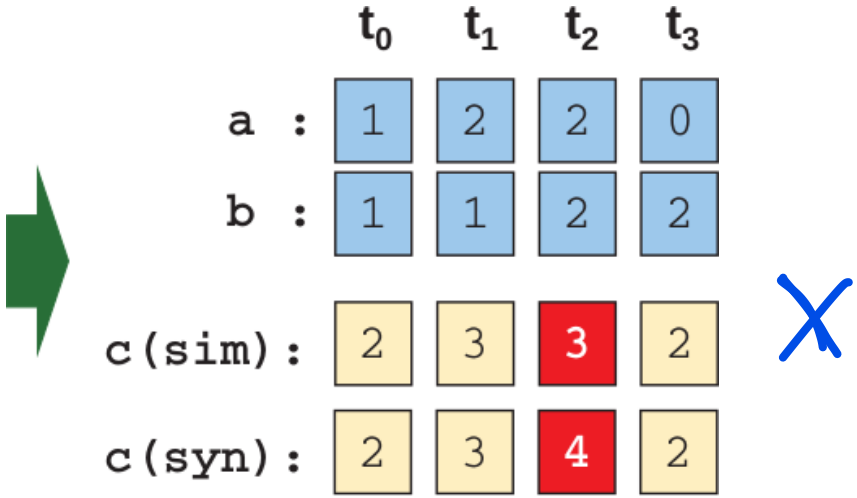
Prozesse



Prozess: Ausführung und Sensitivity-List

- Ein Prozess wird **in der Simulation** nur dann ausfgeführt, wenn sich ein Signal, das in der *Sensitivity-List* des Prozesses steht, ändert
- Achtung: bei der Synthese spielt die *Sensitivity-List* meistens keine Rolle
  - Kombinatorische Prozesse werden so synthetisiert, als ob alle Signale, auf die **lesend** zugegriffen wird, in der *Sensitivity-List* stehen würden
  - Wenn ein Signal vergessen wird, kann das zu Unterschieden zwischen Simulation und Synthese führen!

```
  ...
  signal a, b, c : integer;
begin
  sum : process(a)
  begin
    c <= a + b;
  end process sum;
  ...
```

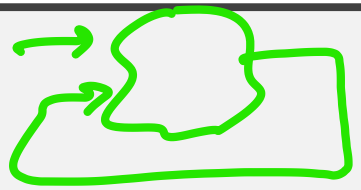


Prozess: Signalverhalten

- Eigenschaften
  - Signale dürfen in Prozessen **nicht** deklariert werden
  - Zuweisungen sind erst am Ende des Prozesses wirksam
  - Bei mehreren Zuweisungen zu einem Signal ist nur die letzte gültig
- Konsequenz: Nutzungsempfehlungen
  - Default-Werte sollten den Signalen in kombinatorischen Prozessen am Prozessanfang zugewiesen werden
  - Signale, denen in einem kombinatorischen Prozess ein Wert zugewiesen wird, sollten nicht im selben Prozess "gelesen" werden

	t_0	t_1	$t_1+\delta$	$t_0+2\delta$	$t_2+\delta$ ... Time
a	1	0	0	0	0
b	1	1	4	1	4
c	~~3~~	2	3	2	3

```
process (a, c)
begin
  c <= 2 + a;
  b <= 1;
  if (c = 2) then
    b <= 4;
    c <= 3;
  end if;
end process;
```



Signalverhalten

Prozess: Variablen

- Variablen sind **keine** Signale, obwohl Operationen und Zuweisungen mit bzw. von/zu Signalen möglich sind, und zwar mit denselben Datentypbeschränkungen wie bei Signalen
- Eigenschaften
  - Variablen sind auf Prozesse beschränkt und auch nur in dem Prozess sichtbar. Die Deklaration erfolgt mit dem Schlüsselwort `variable`
  - Variablen ändern ihren Wert sofort nach der Zuweisung
  - Zuweisungen zu Variablen erfolgen mit dem Symbol `:=`
  - Variablen sollten **nur** verwendet werden, um den Wert eines komplexen Ausdrucks, der mehrfach gebraucht wird, zu speichern

```
process (...)
  variable logic_expr : boolean := false; -- Default-Wert (optional)
begin
  logic_expr := not(X1) and (Y0 nor Z); -- := weil Variable links
  sign1      <= logic_expr;             -- sign1/2 sind Signale
  sign2      <= logic_expr and cond;    -- cond auch (oder Ports)
end process;
```

Variablen

Prozess: Konstrukten

- In VHDL gibt es 4 Strukturen, mit denen die Ausführung von Anweisungen im Prozess gesteuert werden kann
  - Bedingte Ausführung von Anweisungen: `if...then`
  - Bedingte Ausführung von Anweisungen mit Alternativen: `if...then...else` bzw `if...then...elsif`
  - Case-Anweisungen
  - Loops: wiederholte Ausführung einer oder mehrerer Anweisungen: `while...loop` bzw. `for...loop`

```
process (sList)
begin
  if cond1 then
    ...
    case cond2 is
      when val1 => ...
      when val2 => ...
      when others =>
        for i in 1 to 4 loop
        ...
        end loop;
    end case;
  else -- not (cond1)
    while cond3 loop
      ...
    end loop;
  end if;
end process;
```

Prozess: `If-then-else`-Anweisungen

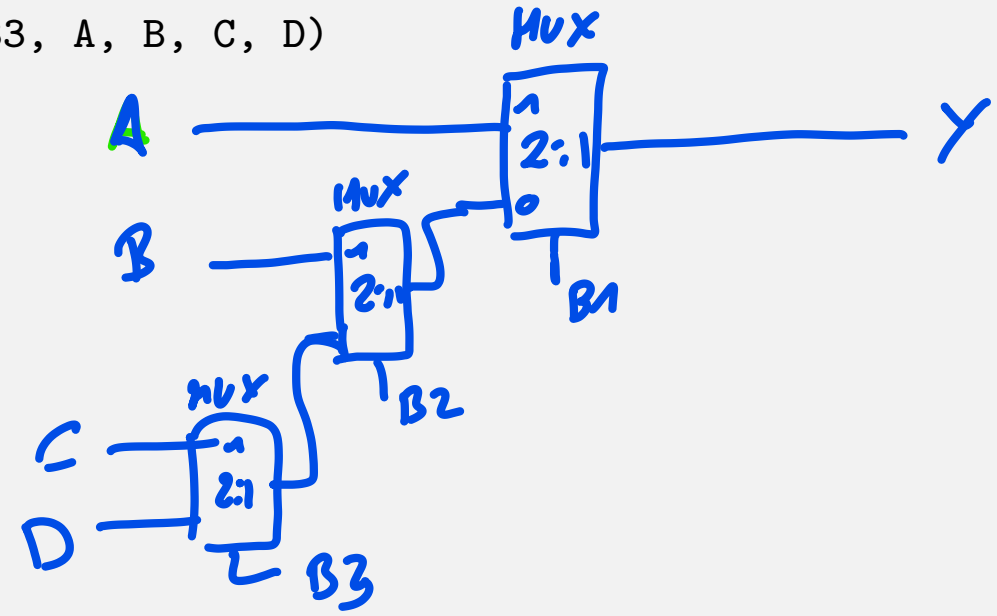
- Die `if-then-else`-Anweisung erlaubt, ähnlich wie die bedingte nebenläufige Signalzuweisung, eine Abfrage mehrerer Bedingungen in den `elsif`-Zweigen und erzeugt damit eine Rangfolge der Bedingungen (**Prioritätsencoder**)
- Innerhalb jeder Verzweigung können weitere unbedingte Signalzuweisungen und sequenzielle Anweisungen, also insbesondere auch weitere `if-then-else`-Anweisungen in beliebiger Veschachtelungstiefe verwendet werden.
- Syntax:

```
if condition then
  sequential_statements
elsif condition then
  sequential_statements
else
  sequential_statements
end if;
```

Prozess: `If-then-else`-Anweisungen (Beispiel I)

```
entity IF_TEST is
  port ( B1, B2, B3 : in std_ulogic;    -- Bedingungssignale
         A, B, C, D : in std_ulogic;    -- Eingangssignale
         Y          : out std_ulogic);  -- AUsgangssignale
end IF_TEST;
architecture RTL of IF_TEST is
begin
  P1 : process(B1, B2, B3, A, B, C, D)
  begin
    if B1 = '1' then
      Y <= A;
    elsif B2 = '1' then
      Y <= B;
    elsif B3 = '1' then
      Y <= C;
    else
      Y <= D;
    end if;
  end process P1;
end architecture RTL;
```

if-beispiel 1

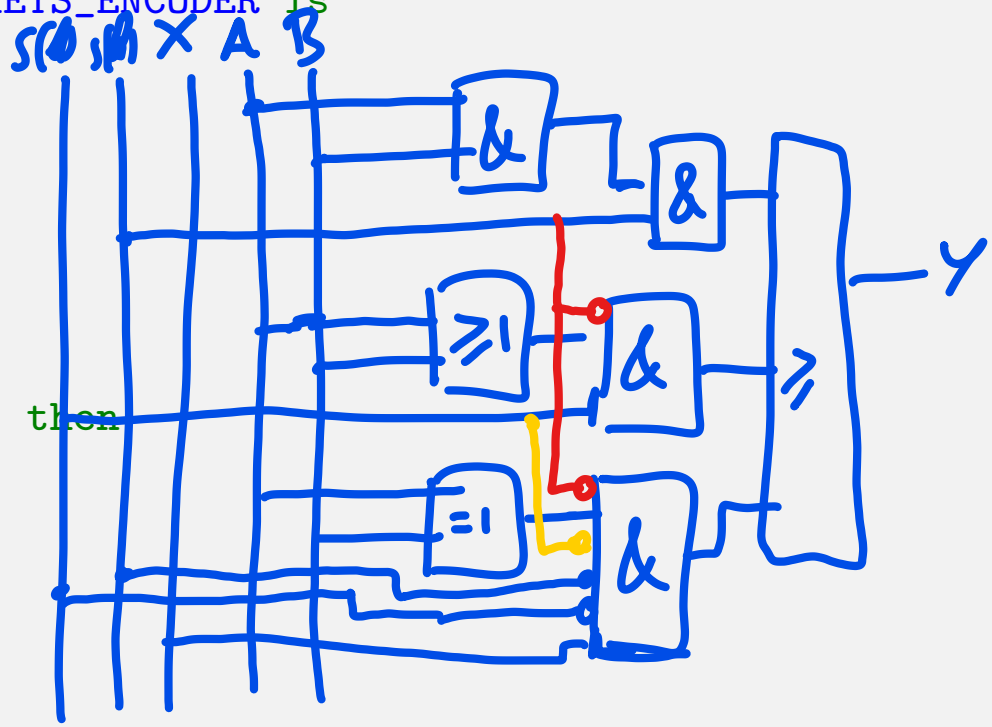


Prozess: `If-then-else`-Anweisungen (Beispiel II)

```
entity PRIORITAETS_ENCODER is
  port ( A, B  : in std_ulogic;
         X     : in std_ulogic;
         S     : in std_ulogic_vector(1 downto 0);
         Y     : out std_ulogic);
end PRIORITAETS_ENCODER;
architecture RTL of PRIORITAETS_ENCODER is
begin
  P1 : process(S, A, B, X)
  begin
    if S(0) = '1' then
      Y <= A and B;
    elsif S(1) = '1' then
      Y <= A or B;
    elsif (S="00" and X='1') then
      Y <= A xor B;
    else
      Y <= '0';
    end if;
  end process P1;
end architecture RTL;
```

S(0) = 1
S(1) = 1

if-Beispiel 2



Prozess: Case-Anweisungen

- Ähnlich wie die switch/case-Anweisung in C existiert in VHDL die case-Anweisung, die eine gleichberechtigte Mehrfachverzweigung zulässt

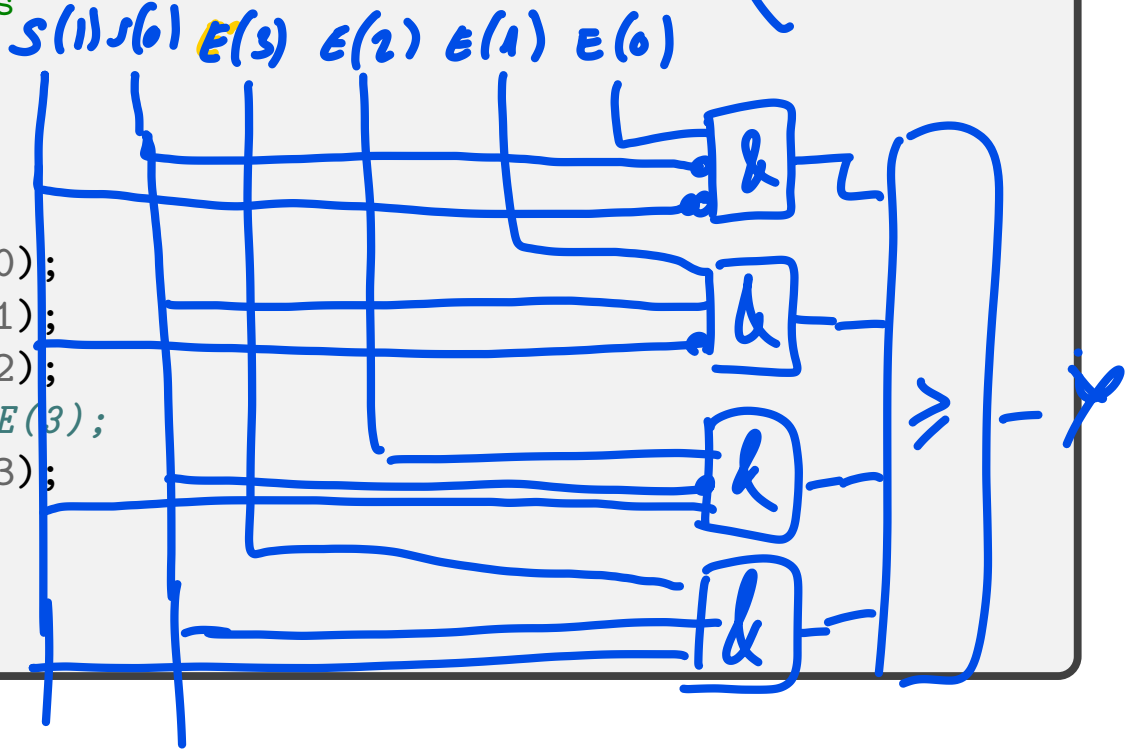
```
case Kontrollausdurck is
  when Testausdruck_1 => sequenzielle Anweisungen;
  when Testausdruck_2 => sequenzielle Anweisungen;
  ...
  when others => sequenzielle Anweisungen; -- oder "when others => null;"
end case;
```

- Alle Testausdrücke müssen sich gegenseitig ausschließen, d.h. keine Testausdruck darf mehr als einmal in eine when-Verzweigung
- Alle möglichen Werte des Kontrollausdrucks müssen in den when-Zweigen berücksichtigt sein → when others.
- Null-Anweisung wenn keine Aktion ausgeführt wird
- Case-Anweisung eignet sich besonders für Zustandsautomaten

Prozess: Case-Anweisungen (Beispiel I)

```
entity MUX4X1 is
  port( S : in std_ulogic_vector(1 downto 0);
        E : in std_ulogic_vector(3 downto 0);
        Y : out std_ulogic);
end MUX4X1;
architecture RTL of MUX4X1 is
begin
  MUXPROC: process(S, E)
  begin
    case S is
      when "00"    => Y <= E(0);
      when "01"    => Y <= E(1);
      when "10"    => Y <= E(2);
--      when "11"    => Y <= E(3);
      when others => Y <= E(3);
    end case;
  end process MUXPROC;
end architecture RTL;
```

S(1) S(0)



case-Beispiel 1

Prozess: Case-Anweisungen (Beispiel II)

```
entity VOLLADD is
  port( C_i, A_i, B_i : in std_ulogic;
        S_out, C_out  : out std_ulogic);
end VOLLADD;
architecture VOLLADD_TAB of VOLLADD is

begin
  SUMPROC: process(C_in, A_in, B_in)
    variable temp_out : std_ulogic_vector(1 downto 0);
  begin
    case C_i & B_i & A_i is
      when "000"              => temp_out := "00";
      when "011"|"101"|"110"  => temp_out := "10";
      when "111"              => temp_out := "11";
      when others             => temp_out := "01";
    end case;
    S_out <= temp_out(0);
    C_out <= temp_out(1);
  end process SUMPROC;
end architecture VOLLADD_TAB;
```

case-Beispiel 2

C_i	B_i	A_i	C_o	S_o
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Prozess: `for`-Anweisungen

- In der `for`-Schleife ist die Anzahl der Schleifendurchläufe durch einen Laufindex vorgegeben, der zwischen zwei Zahlenwerten variiert.
- Die Synthesewerkzeuge fordern, dass diese Grenzwerte statisch sein müssen, womit die Anzahl der Schleifendurchläufe während der Elaboration des VHDL-Codes feststehen muss.

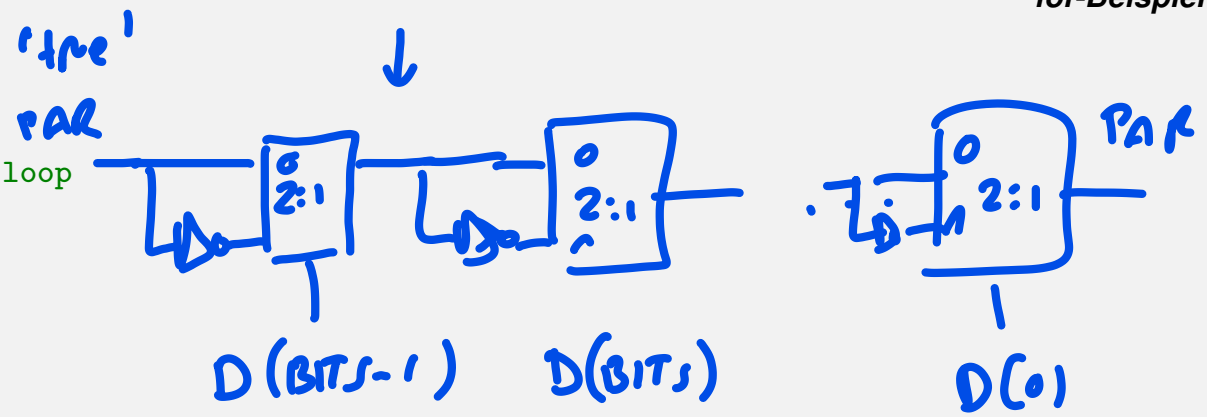
```
Bezeichner1: for schleifenindex in unter_grenze to obere_grenze loop
  sequenzielle Anweisungen
end loop Bezeichner1;
Bezeichner2: for schleifenindex in obere_grenze downto unter_grenze loop
  sequenzielle Anweisungen
end loop Bezeichner2;
```

- Der `Bezeichner` is optional.
- Der `Schleifenindex` muss nicht deklariert werden

Prozess: `for`-Anweisungen (Beispiel)

```
entity PARITAET is
  generic (BITS : integer := 4);
  port (D     : in std_ulogic_vector(BITS-1 downto 0);
        GERADE : out std_ulogic);
end PARITAET;
architecture RTL of PARITAET is
begin
  PARGEN : process (D)
    variable PAR : boolean;
  begin
    PAR := true;
    for I in BITS-1 downto 0 loop
      if (D(I) = '1') then
        PAR := not PAR;
      end if;
    end loop;
    if PAR then
      GERADE <= '1';
    else
      GERADE <= '0';
    end if;
  end process PARGEN;
end architecture RTL;
```

for-Beispiel



GERADE <= PAR;

Prozess: `while`-Anweisungen

- In der `while`-Schleife wird die Schleifenausführung hingegen durch das Eintreten einer Abbruchbedingung beendet, die erst während der Laufzeit eintritt
- Die Anzahl der Schleifendurchläufe ist somit nicht statisch, sondern wird während der Ausführung auf der Zielhardware bestimmt.

```
while schleifenbedingung loop
  sequenzielle Anweisungen
end loop;
```

- Die Synthese von Prozessen mit dem `while`-Konstrukt ist im Synthesestandard 1076.6 nicht vorgesehen.
- **Rein kombinatorische Logik lässt sich mit der `while`-Schleife nicht realisieren!!**
  $\rightarrow$ Sektion: Simulation und Testbench

While-Anweisungen

Prozess: `Don't-Care`-Werte

- Die `Don't-Care`-Werte) können mit den Datentypen `std_logic` und `std_ulogic` verwendet werden
- `Don't-Care`-Werte als Ausgangssignal
  - Damit bleibt es dem Synthesewerkzeug überlassen, die Werte so mit '1' oder '0' zu belegen, dass der Hardwareaufwand minimiert wird.
- `Don't-Care`-Werte als Eingangssignal
  - Nun bedeutet ein Don't-Care-Werte, dass das Eingangssignal entweder `'0'` oder `'1'` ist.
  - Es dient somit einer Abkürzung von Wahrheits- bzw. Arbeitstabellen, da nicht mehr alle Eingangssignalkombinationen individuell aufgeführt werden müssen.

Prozess: Don't-Care-Werte als Ausgangssignal

Binary to Decimal

A ⇐ E;

```
-- Binaer-BCD Umsetzer mit Wahrheitstabelle
-----------------------------------------
P1: process(E)
begin
  case E is
    when x"0"    => A <= x"0";
    when x"1"    => A <= x"1";
    when x"2"    => A <= x"2";
    when x"3"    => A <= x"3";
    when x"4"    => A <= x"4";
    when x"5"    => A <= x"5";
    when x"6"    => A <= x"6";
    when x"7"    => A <= x"7";
    when x"8"    => A <= x"8";
    when x"9"    => A <= x"9";
    when others  => A <= "----";
--  when others => A <= x"0";
  end case;
end process P1;
```

E(3)	E(2)	E(1)	E(0)	A(3)	A(2)	A(1)	A(0)
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1
0	1	1	0	0	1	1	0
0	1	1	1	0	1	1	1
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	1
1	0	1	0	0	0	0	0
1	0	1	1	0	0	0	0
⋮				⋮			

Prozess: Don't-Care-Werte als Eingangssignal

```
case E is     -- <- ordinary case-statement "case"
  when "0000"                                  => A <="100";
  when "0011"                                  => A <="000";
  when "0100" | "0101" | "0110" | "0111"       => A <="101";
  when "1000" | "1100"                         => A <="001";
  when "1001" | "1011" | "1101" | "1111"       => A <="100";
  when others                                  => A <="111";
end case;
```

01--

1--1

Prozess: Don't-Care-Werte als Eingangssignal

```
case E is     -- <- ordinary case-statement "case"
  when "0000"                                  => A <="100";
  when "0011"                                  => A <="000";
  when "0100" | "0101" | "0110" | "0111"  => A <="101";
  when "1000" | "1100"   => 1-00               => A <="001";
  when "1001" | "1011" | "1101" | "1111"  => A <="100";
  when others                                  => A <="111";
end case;
```

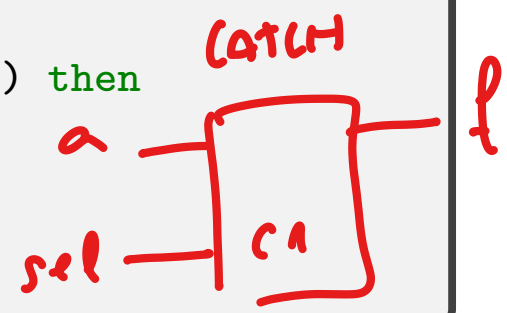
Don't-Care Beispiele

```
case? E is    -- <- matching case-statement "case?"
  when "0000" => A <="100";
  when "0011" => A <="000";
  when "01--" => A <="101";
  when "1-00" => A <="001";
  when "1--1" => A <="100";
  when others => A <="111";
end case?;
```

Pitfalls: Unvollständige `if/case`-Konstrukte

- Sind `if-then-else`- bzw. `case`-Konstrukte unvollständig, werden die Ausgangswerte für die nicht spezifizierten Zustände gehalten
  - pegelgesteuerte Flip-Flops werden synthetisiert, die meistens Probleme verursachen
- Entweder wird dem Ausgangssignal ein spezifikationskonformer Wert gegeben, oder, wie bei `std_ulogic` möglich, ein `don't care`

```
...
  signal sel, a, f : std_ulogic;
begin
  process (sel, a)
  begin

    if (sel = '1') then
      f <= a;
    end if;
    end process;
...
```



Pitfalls: Unvollständige `if/case`-Konstrukte

- Sind `if-then-else`- bzw. `case`-Konstrukte unvollständig, werden die Ausgangswerte für die nicht spezifizierten Zustände gehalten
  - pegelgesteuerte Flip-Flops werden synthetisiert, die meistens Probleme verursachen
- Entweder wird dem Ausgangssignal ein spezifikationskonformer Wert gegeben, oder, wie bei `std_ulogic` möglich, ein `don't care`

```
...
  signal sel, a, f : std_ulogic;
begin
  process (sel, a)
  begin

    if (sel = '1') then
      f <= a;
    end if;
    end process;
...
```

```
...
  signal sel, a, f : std_ulogic;
begin
  process (sel, a)
  begin
    f <= '-';
    if (sel = '1') then
      f <= a;
    end if;
  end process;
...
```

a) f<=a;

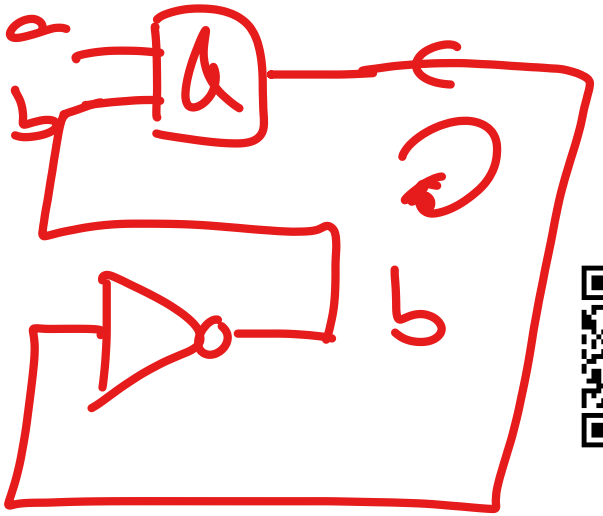
f<=a;

Pitfalls 1

Pitfalls: Kombinatorische Rückkopplungen

- Komb. Rückkopplungen können entstehen, wenn das Verhalten von Signalen in Prozessen nicht richtig interpretiert wird
  - Ausgangssignale wurden im selben Prozess gelesen
  - Unter bestimmten Bedingungen kommt es dann zu einer Änderung des ursprünglich als Eingang interpretierten Signals, was bei komplexen Prozessen schon unbemerkt passieren kann
- Hardware-Oszillatoren können dadurch entstehen

```
process (a, b)
begin
  ...
  c <= a and b;
end process;
process (c)
begin
  ...
  b <= not c;
end process;
```



Pitfalls 2

Literatur

- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.